PROJECT 6.

AI ASSISTANCE RESEARCH & DESCRIPTION

для проєкту
«Каталог фільмів з адаптивними рекомендаціями»

Роботу виконала студентка
1 курсу магістратури
комп’ютерних наук
Група інформатика -1
Голубцова Орина Євгенівна

Київ – 2026

6 AI ASSISTANCE RESEARCH & DESCRIPTION

У цьому розділі досліджено використання OpenAI Codex під час створення проєкту «Каталог фільмів з адаптивними рекомендаціями». ШІ застосовувався як допоміжний інструмент: для структурування вимог і проєктних рішень, підготовки та пояснення окремих фрагментів реалізації, аналізу помилок, формування тестових сценаріїв, рефакторингу, підготовки розгортання, навантажувального тестування та оформлення звітності. Остаточні рішення, запуск команд, перевірка результатів і виправлення неточностей виконувалися студенткою.

Структуру аналізу адаптовано за навчальним прикладом звіту [1]. Для кожного етапу наведено мету використання ШІ, репрезентативні реконструйовані запити, витрати часу, орієнтовну частку пропозицій, що потребували суттєвого виправлення, а також переваги, недоліки й порівняння з повністю ручним виконанням. Реконструйовані запити відтворюють зміст реальної взаємодії, але не претендують на дослівний журнал усіх повідомлень.

6.1 Модель і методика оцінювання

6.1.1 Використана модель

У роботі використовувався OpenAI Codex. Інші моделі не застосовувалися. Codex допомагав аналізувати код і документи в межах робочого каталогу, пропонувати зміни, пояснювати помилки PowerShell, Python, Node.js і Docker, а також перевіряти узгодженість технічних результатів зі звітами. Самостійна робота залишалася базовим способом виконання завдання: визначати функціональність, оцінювати доцільність рекомендацій ШІ, запускати програму й тести та приймати остаточні рішення.

6.1.2 Критерії оцінювання

1. Коректність -відповідність вимогам, вихідному коду, фактичним результатам запуску та звітним даним.
2. Корисність -здатність рекомендації скоротити пошук рішення або зробити пояснення зрозумілішим.

3. Потреба у виправленні -наявність логічної, програмної або фактологічної помилки, яку потрібно було виправити вручну. Стилістичні правки до цього показника не включалися.
4. Час роботи з ШІ -формулювання запиту, читання відповіді, адаптація пропозиції та перевірка результату.
5. Відтворюваність -можливість підтвердити висновок повторним запуском тесту, збірки, сценарію або перевіркою файлу.

Таблиця 6.1 – Ролі та способи перевірки

Компонент	Роль Codex	Ручна перевірка
Документація	Структурування, формулювання, пошук суперечностей	Звіряння з вимогами та фактичним станом проєкту
Програмний код	Чернетки змін, пояснення помилок, рефакторинг	Перегляд різниці, локальний запуск і тести
Тестування	Сценарії та інтерпретація результатів	Запуск pytest, coverage, Vite build і браузерних перевірок
Продуктивність	Підготовка сценарію та аналіз метрик	30-хвилинний запуск і перевірка JSON-звіту
Маркетинг	Структурування сегментів, каналів і KPI	Вилучення непідтверджених чисел і узгодження з прототипом

6.2 Етап 1. Requirements

6.2.1 Застосування ШІ

На етапі Requirements Codex допоміг упорядкувати бізнес-, користувацькі, функціональні та нефункціональні вимоги. Особлива увага приділялася простежуваності функцій каталогу, пошуку, особистих списків, оцінювання за шкалою від 1 до 5, рекомендацій Top-K та адміністративних операцій. ШІ також використовувався для пошуку нечітких формулювань і суперечностей між вимогами та майбутньою реалізацією.

6.2.2 Репрезентативні запити

Таблиця 6.2 – Приклади запитів на etani Requirements

№	Реконструйований запит	Результат
1	Сформуй ієрархію BR, UR, FR і NFR для каталогу фільмів з адаптивними рекомендаціями. Не додавай функцій, яких немає в описі проєкту.	Чернетка структури вимог, яку було відредаговано й узгоджено зі звітом.
2	Перевір простежуваність функцій: пошук і фільтри, оцінка 1–5, Favorite, Watchlist, Watched, історія та Top-K рекомендації.	Виявлено місця, де потрібно було точніше визначити ролі користувача й адміністратора.
3	Перевір NFR: безпека, продуктивність, сумісність, зручність і критерії приймання. Познач, що ще не підтверджено тестами.	Критерії були відокремлені від майбутніх або ще не перевірених цілей.

6.2.3 Час і потреба у виправленні

Таблиця 6.3 – Оцінка роботи на etani Requirements

Показник	Значення	Пояснення
Загальний час етапу	180 хв	Аналіз предметної області, формулювання й перевірка вимог.
Час роботи з Codex	75 хв	Запити, аналіз відповідей, редагування й звіряння.
Суттєві виправлення	орієнтовно 10% пропозицій	Переважно уточнення критеріїв приймання та усунення надмірних припущень.

6.2.4 Переваги, недоліки та ручна альтернатива

Таблиця 6.4 – Порівняння підходів на етапі Requirements

Переваги Codex	Недоліки й ризики	Порівняння з ручною роботою
Швидке структурування великої кількості вимог; пошук пропущених зв'язків; однакова термінологія.	Може запропонувати функції поза межами проєкту або сформулювати критерії без доказової бази.	Ручний аналіз краще враховує контекст завдання, але потребує більше часу на систематизацію.
Допомога у формуванні перевірюваних критеріїв.	Не розрізняє автоматично реалізовану функцію та заплановану ціль.	Найкращий результат отримано після ручного відбору й уточнення кожної вимоги.

6.3 Етап 2. Design

6.3.1 Застосування ШІ

На етапі Design ШІ використовувався для перевірки структури бази даних, ролей сутностей і зв'язків між ними, а також для формалізації алгоритму адаптивних рекомендацій. Codex допоміг описати побудову зваженого профілю користувача, cosine similarity, cold start, пояснення рекомендацій і функцію схожих фільмів. Архітектурні рішення перевірялися на відповідність обраному стеку FastAPI, SQLAlchemy, React/TypeScript і PostgreSQL 16.

6.3.2 Репрезентативні запити

Таблиця 6.5 – Приклади запитів на етапі Design

№	Реконструйований запит	Результат
1	Перевір схему User, Movie, Genre, Person, Rating і UserInteraction: PK, FK, кардинальності, унікальні обмеження та каскадне видалення.	Уточнено відповідальність таблиць поточного стану та історії взаємодій.
2	Перевір алгоритм weighted content profile + cosine similarity. Ваги: rating 0.60, Favorite 0.25,	Підтверджено суму ваг і послідовність розрахунку; додано

№	Реконструйований запит	Результат
	Watched 0.10, Watchlist 0.05; перетворення rating max((r−3)/2, 0).	обробку порожнього профілю.
3	Запропонуй межі модулів main.py, interactions.py, recommendation.py і models.py без зміни функціональних вимог.	Визначено поділ маршрутів, транзакцій взаємодій, рекомендаційної логіки та моделей.

6.3.3 Час і потреба у виправленні

Таблиця 6.6 – Оцінка роботи на etani Design

Показник	Значення	Пояснення
Загальний час етапу	240 хв	Схема даних, API, алгоритм рекомендацій та модульна структура.
Час роботи з Codex	95 хв	Перевірка альтернатив, формул і меж модулів.
Суттєві виправлення	орієнтовно 12% пропозицій	Уточнення транзакцій, cold start і відповідності схеми реальним моделям.

6.3.4 Переваги, недоліки та ручна альтернатива

Таблиця 6.7 – Порівняння підходів на etani Design

Переваги Codex	Недоліки й ризики	Порівняння з ручною роботою
Дає кілька архітектурних варіантів; швидко перевіряє арифметичну узгодженість ваг; пояснює алгоритм зрозумілою мовою.	Без коду та даних може запропонувати надмірно складну схему або загальний алгоритм без потрібних граничних випадків.	Ручне проєктування забезпечує глибше розуміння, а Codex корисний як інструмент другої перевірки.
Допомагає пов’язати вимоги, API та моделі даних.	Не замінює перевірку міграцій і транзакцій у реальній БД.	Фінальну схему було прийнято тільки після звіряння з

Переваги Codex	Недоліки й ризики	Порівняння з ручною роботою
		SQLAlchemy-моделями й міграцією Alembic.

6.4 Етап 3. Development and Testing

6.4.1 Застосування ШІ

Під час розробки Codex допомагав реалізовувати та перевіряти REST API, авторизацію, ролі, транзакційне оновлення взаємодій і рекомендаційний алгоритм. Для клієнта ШІ використовувався під час удосконалення навігації, розділення входу та реєстрації на вкладки, додавання постерів, описів, акторів, режисерів і рейтингових даних. Окремо аналізувалися помилки HTTP 422 та невідповідності між даними backend і відображенням frontend.

На етапі тестування Codex допоміг підготувати unit- та integration-сценарії, пояснити звіт coverage, перевірити production build і автоматизувати п’ять браузерних сеансів. Автоматизовані сеанси використовувалися для технічної перевірки інтерфейсу й не подавалися як дослідження з п’ятьма реальними учасниками.

6.4.2 Репрезентативні запити

Таблиця 6.8 – Приклади запитів на етапі Development and Testing

№	Реконструйований запит	Результат
1	Проаналізуй HTTP 422 під час входу або реєстрації у FastAPI. Звір формат JSON, Pydantic-схему й URL frontend.	Виправлено невідповідність запиту; повідомлення про помилки стало зрозумілішим.
2	Покращи React-інтерфейс: вкладки входу/реєстрації, картки з постерами, адаптивна сітка, особисті дії та зрозумілий стан кнопок.	Оновлено UI без зміни вимог до шкали оцінювання 1–5.

№	Реконструйований запит	Результат
3	Додай тести interactions.py і recommendation.py: поточний стан, історія, ваги, cosine, cold start, пояснення та similar.	Розширено набір тестів і перевірено критичну логіку.
4	Поясни coverage report: 130 statements, 6 missing, 92%.	Показники у звіті були приведені у відповідність до фактичного виводу.

6.4.3 Час і потреба у виправленні

Таблиця 6.9 – Оцінка роботи на етапі Development and Testing

Підетап	Загальний час	Час роботи з Codex	Суттєві виправлення
Development	480 хв	205 хв	орієнтовно 15% пропозицій
Testing	300 хв	120 хв	орієнтовно 10% пропозицій

6.4.4 Фактично підтверджені результати

Таблиця 6.10 – Результати перевірки реалізації

Перевірка	Фактичний результат	Висновок
Backend pytest	17 тестів пройдено; 1 попередження про застарілий імпорт TestClient	Основні сценарії backend працюють; попередження не є падінням тесту.
Branch coverage	interactions.py -100%; recommendation.py -92%; разом -94%	Критична логіка покрита тестами, але не повністю.
Frontend production build	Vite успішно зібрав 29 модулів	TypeScript/React-клієнт збирається для розгортання.
Автоматизовані UI-сеанси	5 із 5 технічних сеансів виконали пошук, Watchlist, оцінку й відкриття рекомендацій	Це автоматизована перевірка, а не human usability study.

Перевірка	Фактичний результат	Висновок
Облікові записи	Перевірено окремі демонстраційні ролі user та admin	Маршрути й інтерфейс перевіряють роль користувача.

6.4.5 Переваги, недоліки та ручна альтернатива

Таблиця 6.11 – Порівняння підходів на etani Development and Testing

Переваги Codex	Недоліки й ризики	Порівняння з ручною роботою
Швидко локалізує типові помилки; узгоджує API і UI; пропонує тестові граничні випадки.	Згенерований код може не врахувати наявні моделі, версії бібліотек або стиль проєкту.	Ручна реалізація повільніша, але дає повний контроль; пропозиції ШІ потребують перегляду різниці й запуску тестів.
Допомагає тлумачити coverage та відокремлювати warning від error.	Може перебільшити результат, якщо отримує лише частину журналу виконання.	Фактичні числа бралися тільки з термінала та сформованих звітів.

6.5 Етап 4. Refactoring, Deployment and Delivery

6.5.1 Застосування ШІ

Під час рефакторингу Codex використовувався для розділення відповідальностей між main.py, interactions.py і recommendation.py, усунення дублювання та збереження поведінки API. На етапі розгортання ШІ допоміг проаналізувати Docker Compose, підготувати .dockerignore, локалізувати помилку копіювання node_modules у frontend-образ і пояснити різницю між локальним SQLite та цільовим PostgreSQL 16.

Для delivery і performance testing було підготовлено сценарій створення тестового каталогу, запуску окремого API на порту 8001, формування токенів і 30-хвилинного навантаження з 20 віртуальними користувачами. Codex також допоміг виправити обмеження набору даних 20 018 записів замість максимально допустимих 20 000 та замінити недостатню затримку запуску API на перевірку готовності.

6.5.2 Репрезентативні запити

Таблиця 6.12 – Приклади запитів на etani Refactoring, Deployment and Delivery

№	Реконструйований запит	Результат
1	Проаналізуй Docker build error: cannot replace directory node_modules/@types/react with file. Збережи package.json і виключи локальні залежності з build context.	Створено frontend/.dockerignore; Docker-контекст очищено від node_modules.
2	Підготуй 30-хвилинний load test: 20 virtual users, до 20 000 movies, search і recommendations, p95, HTTP 5xx та JSON-звіт.	Сценарій виконує вимірювання й зберігає відтворюваний звіт.
3	API на порту 8001 не встигає запуститися. Заміни фіксоване очікування на readiness polling і покажи реальну помилку процесу.	Запуск став стабільнішим; тест завершився повним 30-хвилинним прогоном.

6.5.3 Час і потреба у виправленні

Таблиця 6.13 – Оцінка роботи на etani Refactoring, Deployment and Delivery

Підетап	Загальний час	Час роботи з Codex	Суттєві виправлення
Refactoring, deployment, delivery	420 хв	165 хв	орієнтовно 12% пропозицій
Performance testing	240 хв	95 хв	орієнтовно 11% пропозицій

6.5.4 Результати навантажувального тесту

Таблиця 6.14 – Фактичні метрики 30-хвилинного локального тесту

Метрика	Результат	Критерій	Статус
Тривалість і навантаження	1800,05 с; 20 virtual users	1800 с; 20 users	виконано

Метрика	Результат	Критерій	Статус
Успішні HTTP-відповіді	142 776 HTTP 200; 0 помилок	без HTTP 5xx	виконано
Search P95	855,86 мс	≤ 800 мс	не виконано
Recommendations P95	870,67 мс	≤ 1200 мс	виконано
Persistence check	80/80 записів; 0 невідповідностей	усі записи збережені	виконано окремим сценарієм
Середовище	Windows + SQLite	PostgreSQL 16; 2 vCPU; 4 GB RAM	цільове середовище не перевірено

Результат не приховує невиконаний критерій Search P95. Тест підтвердив стабільність локального прототипу без серверних помилок, але не є повною верифікацією NFR у цільовому PostgreSQL-середовищі. Перед продуктивним розгортанням необхідно повторити вимірювання на визначеній конфігурації ресурсів.

6.5.5 Переваги, недоліки та ручна альтернатива

Таблиця 6.15 – Порівняння підходів на etani Refactoring, Deployment and Delivery

Переваги Codex	Недоліки й ризики	Порівняння з ручною роботою
Допомагає читати журнали Docker і автоматизувати повторювані перевірки; пропонує безпечні кроки діагностики.	Може припустити наявність Docker, Playwright або іншої залежності, якої немає в середовищі.	Ручна діагностика точніша щодо конкретного ПК, але ШІ скорочує перебір типових причин.
Структурує метрики й критерії успіху навантажувального тесту.	Без контролю може представити локальний результат як доказ відповідності production NFR.	Середовище, тривалість, невиконані цілі й обмеження були зафіксовані вручну.

6.6 Етап 5. Marketing Strategy

6.6.1 Застосування ШІ

Для маркетингової стратегії Codex допоміг структурувати ціннісну пропозицію сервісу, сегменти користувачів, канали залучення, freemium- і B2B-напрями, KPI, ризики та план розвитку. Оскільки проєкт є навчальним прототипом, непідтверджені ринкові цифри не подавалися як встановлені факти, а прогнозні показники були відокремлені від результатів технічного тестування.

6.6.2 Репрезентативні запити

Таблиця 6.16 – Приклади запитів на etani Marketing Strategy

№	Реконструйований запит	Результат
1	Сформуй маркетингову стратегію для каталогу фільмів: аудиторії, ціннісна пропозиція, freemium, B2B, канали, KPI та ризики. Не вигадуй ринкову статистику.	Отримано структурований план, адаптований до навчального прототипу.
2	Відокрем фактичні показники продукту від прогнозів і майбутніх цілей. Використовуй лише підтверджені технічні результати.	У стратегії позначено, які метрики вже виміряні, а які потребують пілотного запуску.

6.6.3 Час і потреба у виправленні

Таблиця 6.17 – Оцінка роботи над звітністю та маркетинговою стратегією

Показник	Значення	Пояснення
Загальний час	360 хв	Аналіз, маркетингова стратегія, звіти та остаточне редагування.
Час роботи з Codex	135 хв	Структурування, чернетки, перевірка чисел і мовне редагування.

Показник	Значення	Пояснення
Суттєві виправлення	орієнтовно 9% пропозицій	Вилучення непідтверджених ринкових оцінок і надто загальних KPI.

6.6.4 Переваги, недоліки та ручна альтернатива

Таблиця 6.18 – Порівняння підходів на emani Marketing Strategy

Переваги Codex	Недоліки й ризики	Порівняння з ручною роботою
Швидко структурує аудиторії, канали, KPI та ризики; допомагає пов’язати технічні переваги з цінністю для користувача.	Може генерувати правдоподібні, але неперевірені статистичні дані або надто оптимістичні прогнози.	Ручна робота краще враховує реальний ринок; ШІ корисний для першої структури й перевірки повноти.
Полегшує мовне редагування та узгодження термінів між звітами.	Не замінює інтерв’ю, аналіз конкурентів і реальні дані про конверсію.	Фінальна стратегія подає прогнозні KPI як майбутні цілі, а не досягнуті результати.

6.7 Загальне порівняння та ручна перевірка

6.7.1 Розподіл часу

Таблиця 6.19 – Орієнтовні витрати часу за видами робіт

Вид роботи	Загальний час	Час роботи з Codex	Основна роль Codex
Requirements	180 хв	75 хв	Структурування й перевірка вимог
Design	240 хв	95 хв	Перевірка схеми й алгоритму
Development	480 хв	205 хв	Чернетки змін і діагностика
Testing	300 хв	120 хв	Сценарії, coverage, UI-перевірка
Refactoring, deployment, delivery	420 хв	165 хв	Рефакторинг, Docker і

Вид роботи	Загальний час	Час роботи з Codex	Основна роль Codex
			документація запуску
Performance testing	240 хв	95 хв	Сценарій навантаження й аналіз метрик
Reporting and marketing	360 хв	135 хв	Структурування та редагування
Разом	2220 хв	890 хв	Допоміжна роль із ручною перевіркою

Наведені значення є ретроспективною оцінкою навчальної роботи. Вони показують, що взаємодія з Codex була регулярною, але значна частина часу припадала на самостійне визначення задач, реалізацію, запуск команд, перевірку й виправлення. Час спілкування з ШІ не ототожнюється з автоматично створеним результатом, оскільки до нього включено читання, адаптацію та верифікацію.

6.7.2 Виявлені обмеження та виправлення

Таблиця 6.20 – Приклади ручної перевірки відповідей Codex

Ситуація	Початкова проблема	Ручна перевірка або виправлення
Playwright	Сценарій запускався не з кореня проєкту або залежність була відсутня.	Перевірено робочий каталог, створено package.json, встановлено Playwright і Chromium.
Coverage	Існував ризик назвати 92% повним покриттям.	У звіт внесено 130 statements, 6 missing і фактичні 92% для recommendation.py.
Usability	Автоматизовані сеанси могли бути помилково подані як п’ять реальних учасників.	У звіті явно зазначено, що human usability study не проводилося.

Ситуація	Початкова проблема	Ручна перевірка або виправлення
Docker build	Локальний node_modules потрапив у контекст і спричинив конфлікт типів файлів.	Додано frontend/.dockerignore та повторено збірку з очищеним контекстом.
Load-test dataset	Було створено 20 018 фільмів за обмеження 20 000.	Тестову БД перебудовано до 20 000 записів.
API readiness	Фіксованого очікування не вистачало для запуску порту 8001.	Реалізовано опитування готовності API з діагностикою процесу.
Performance conclusion	Результат міг бути узагальнений на цільове середовище.	Збережено позначку Search P95 -not met і обмеження Windows + SQLite.

6.7.3 Що виконувалося самостійно

– вибір теми, цілей, меж і функціональних пріоритетів проєкту;

– прийняття архітектурних рішень і затвердження схеми даних;

– редагування й інтеграція запропонованого коду у власний проєкт;

– запуск сервісів, команд, міграцій, seed, тестів, збірки та навантажувального сценарію;

– перевірка UI, облікових записів, ролей і персональних дій;

– звіряння чисел на скріншотах, у JSON-звітах і таблицях;

– остаточне формулювання висновків та відповідальність за поданий результат.

6.8 Відповідальне використання ШІ

Використання Codex не звільняє автора від перевірки результату. Згенеровані фрагменти не приймалися лише тому, що вони виглядали правдоподібно. Для коду обов’язковими були перегляд змін і запуск тестів; для числових показників -звіряння з терміналом або JSON-звітом; для документації -перевірка відповідності вимогам і реальному стану реалізації.

До звіту не включалися секрети, паролі або персональні дані реальних користувачів. Для демонстрації використовувалися тестові облікові записи.

Паролі зберігаються у вигляді bcrypt-хешів, а доступ до персональних і адміністративних маршрутів контролюється JWT та серверною перевіркою ролі.

Обмеження дослідження також подано відкрито: п’ять автоматизованих браузерних сеансів не замінюють тестування з п’ятьма людьми; навантажувальний тест у Windows + SQLite не замінює перевірку PostgreSQL 16 на заданих ресурсах; невиконаний критерій Search P95 не приховується. Такий підхід дає можливість відтворити результати й коректно спланувати наступні перевірки.

Висновки

OpenAI Codex був корисним допоміжним інструментом протягом усіх етапів проєкту: від Requirements і Design до Development, Testing, Refactoring, Deployment, Delivery та Marketing Strategy. Найбільшу практичну цінність він мав під час структурування матеріалу, пояснення технічних помилок, підготовки тестових сценаріїв, перевірки узгодженості та формування відтворюваних команд.

Водночас результати показали, що відповіді ШІ потребують критичної перевірки. Codex міг не врахувати конкретне середовище, припустити наявність залежностей, перебільшити повноту тестування або змішати автоматизовану перевірку з дослідженням за участю людей. Надійність забезпечувалася ручним переглядом, запуском тестів, звірянням чисел і чесним зазначенням обмежень.

Отже, ШІ підвищив швидкість аналізу й допоміг покращити якість оформлення та технічної перевірки, але не замінив самостійне проєктування, реалізацію, експеримент і відповідальність автора. Підготовлений звіт відображає не лише позитивні результати, а й невиконані критерії та подальші кроки, що робить оцінювання прозорим.